Final Report

NLP-based Scientific Article Suggestion System

Sena ÖZ
Advisor: H. Birkan YILMAZ

June 4, 2026

TABLE OF CONTENTS

1. Introduction 2
1.1. Motivation 2
1.2. Project Objective 2
1.3. Scope 3
2. System Description 3
2.1. Architecture Overview 4
2.2. Backend Pipeline 5
2.3. Data Model 9
2.4. Frontend Application 10
3. System Requirements 14
3.1. Functional Requirements 14
3.2. Non-Functional Requirements 15
4. Research Methodology and Evaluation 15
4.1. Dataset Construction 16
4.1.1. Data Collection 16
4.1.2. Preprocessing 16
4.1.3. Benchmark Dataset for Cited Paper Ranking 16
4.1.4. Pipeline Validation 17
4.2. Text Representation and Similarity Methods 18
4.2.1. Field Ablation 19
4.2.2. Pipeline Component Ablation 20
4.3. Evaluation Metrics 21
4.4. Results 22
4.5. Discussion 22
5. Conclusion and Future Work 23
Bibliography 25

1. Introduction

1.1. Motivation

The volume of academic publications is growing at an unprecedented rate. Researchers at Boğaziçi University (BOUN) and similar institutions face an increasing challenge: staying up to date with newly published work from peer institutions is time-consuming and error-prone when done manually. A researcher may miss a directly relevant paper simply because it was published by an institution they do not routinely monitor.

Existing recommendation platforms such as Semantic Scholar or Google Scholar provide general-purpose suggestions, but they are not tailored to an institution's specific researcher profiles. There is a gap between the *global* stream of publications and the *local* expertise of a given research community.

1.2. Project Objective

This project aims to build an automated, NLP-based paper recommendation system targeted at Boğaziçi University (BOUN) researchers. Given the volume of new publications appearing daily across leading research institutions, the primary objective is to reduce the manual effort required for researchers to discover work that is relevant to their own expertise.

Concretely, the system pursues four goals:

(i) **Automated ingestion** — Continuously monitor a configurable set of academic institutions via the OpenAlex API [4] and ingest newly published papers without manual intervention.
(ii) **Profile-aware matching** — Construct a vector-space representation of each BOUN researcher's expertise from their publication history and rank incoming papers against those profiles using cosine similarity.
(iii) **Two-stage reranking** — Evaluate whether a lightweight LLM reranking stage applied on top of dense retrieval (a two-stage RAG pipeline [3]) yields measurably better hit-rate performance than embedding-only retrieval.
(iv) **Production deployment** — Deliver a fully containerized, end-to-end system — including a REST API, a React-based user interface, and an admin panel — that a real research institution could operate.

The central research question is: *Does LLM-based reranking of embedding-retrieved candidates improve recommendation quality for academic papers, and if so, by how much*

and under what conditions? The full source code, research notebooks, and deployment configuration are publicly available at `https://github.com/senaoz/SWE-599`.

1.3. Scope

The following boundaries define what is and is not covered by this project.

Institutions and time window.. The system monitors seven major research institutions — MIT, Stanford, Harvard, UC Berkeley, University of Michigan, Google, and Google DeepMind — and considers only English-language publications indexed by OpenAlex after 2020. Institutions are configurable at runtime; the seven listed represent the initial seed set used for evaluation.

Researcher profiles.. BOUN researcher profiles are constructed from 4,608 papers authored by BOUN-affiliated researchers and retrieved from OpenAlex. Each profile is represented as the mean embedding of the researcher's publication abstracts; author disambiguation and profile maintenance over time are outside the scope of this project.

Embedding and reranking models.. The offline evaluation compares eight retrieval methods (TF-IDF, BM25, MiniLM, SPECTER2, Gemini text-embedding-004, Llama 3.2, Qwen3, and a Qwen3+MiniLM hybrid). The production system uses Qwen3 for Stage 1 retrieval and Llama 3.2 3B via Ollama for Stage 2 reranking. Fine-tuning any of these models is out of scope.

Evaluation dataset.. Model selection and hit-rate measurements are performed on a hand-curated evaluation set of 50 BOUN-authored papers paired with the papers they cite. The assumption is that a cited paper is, by definition, relevant to the citing researcher's work. Coverage beyond this cite-based proxy (e.g. user studies, click-through data) is out of scope.

Exclusions.. The system does not perform full-text retrieval (only title, abstract, and concept tags are used), does not support non-English publications, does not provide personalisation beyond publication-history profiles, and does not handle real-time paper feeds (ingestion runs on a six-hour schedule).

2. System Description

2.1. Architecture Overview

The system is composed of three layers:

(i) **Research Layer** — Offline Jupyter notebooks for dataset construction, embedding method evaluation, and metric computation. This layer drives model selection decisions for the production system. A dedicated demo notebook (`pipeline_demo.ipynb`) illustrates the full two-stage RAG pipeline end-to-end against live BOUN embeddings.

(ii) **Backend Layer** — A FastAPI [5] service backed by PostgreSQL 16 with the `pgvector` extension that runs the paper ingestion pipeline, computes embeddings, stores matches, and exposes a REST API.

(iii) **Frontend Layer** — A React 19 single-page application that allows users to follow institutions, browse recommended papers, explore researcher profiles, and manage the system via an admin panel.

The full stack is containerized with Docker Compose using the `pgvector/pgvector:pg16` image, which bundles the `pgvector` extension alongside PostgreSQL 16. The backend runs on port 8001; the frontend development server runs on port 5173. Embedding inference is provided via a configurable provider chain (see Section 2.2).

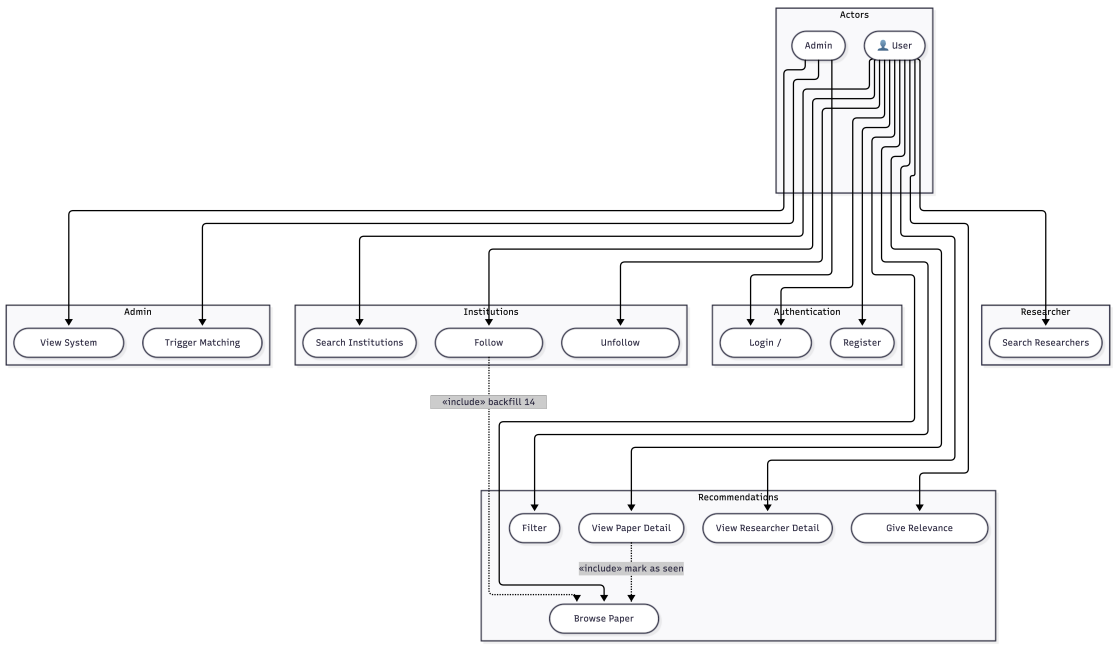


Figure 2.1. Use Case Diagram — actors (Researcher, Admin) and primary system interactions.

2.2. Backend Pipeline

On startup, the backend seeds the `researchers` table from a pre-processed CSV file (`boun.csv`) containing BOUN researcher metadata. Embeddings for each researcher's papers are computed asynchronously using Qwen3 [8] and stored as native `vector(4096)` columns in PostgreSQL via the `pgvector` extension.

An APScheduler job executes every six hours (`JOB_INTERVAL_HOURS = 6`). For each followed institution, the job:

(i) Fetches papers published since the last recorded cursor date via the OpenAlex API.
(ii) Encodes new paper texts (title + abstract + concept tags) with the Qwen3 embedding model (`qwen3-embedding:latest`).
(iii) Computes cosine similarity between each new paper and all stored BOUN researcher paper embeddings (*Stage 1 — Retrieve*).
(iv) Groups hits by researcher and selects the top 10 candidates above a configurable threshold (`MATCH_THRESHOLD = 0.3`).
(v) For the top 10 candidates, invokes Llama 3.2 3B via Ollama with a structured prompt asking whether the researcher's prior work is relevant to the new paper (*Stage 2 — Rerank*).
(vi) Stores final match scores and the matched BOUN paper identifiers in the `paper_researcher_match` table.

If Ollama is unavailable, Stage 2 is bypassed and Stage 1 cosine similarity scores are used directly, ensuring graceful degradation.

Stage 2 prompt design.. The reranking prompt is intentionally minimal to reduce latency and avoid over-constraining a small (3B-parameter) model. Given a new paper and up to 10 of the researcher's most similar prior publications, the model is asked a binary relevance decision with an associated confidence score:

```
Decide if a researcher should receive a recommendation for this new paper.

NEW PAPER:
Title: {title}
Abstract: {abstract[:400]}

RESEARCHER'S RELATED PAPERS:
- {paper_title}: {abstract[:200]}
```

```
...

Answer with ONLY: RELEVANT <score> or IRRELEVANT <score> (score: 0.0-1.0)
Example: RELEVANT 0.85
```

The structured output format (`RELEVANT 0.85` / `IRRELEVANT 0.12`) was chosen because it is reliably parseable from a small model without requiring JSON mode or grammar-constrained decoding. Abstracts are truncated to 400 characters for the new paper and 200 characters per context paper to keep the prompt within the model's context window and maintain consistent latency. If the model's response cannot be parsed, the Stage 1 cosine similarity score is used as a fallback.

Embedding provider fallback.. To support deployments where a local Ollama instance is not available, the system implements a two-tier provider fallback chain: **local Ollama** (tried first) → **Ollama Cloud** (tried if local Ollama is unreachable, using the `OLLAMA_CLOUD_API_KEY` and `OLLAMA_CLOUD_MODEL` environment variables). If both providers fail, the job aborts with an error. The `encode_with_fallback()` function in `backend/services/embedding.py` encapsulates this logic and is used by both the seeder and the matching job.

Native vector storage.. Embeddings are stored as `vector(4096)` columns in PostgreSQL using the `pgvector` extension. This eliminates the need to load all embeddings into Python memory for similarity computation and makes SQL-level approximate nearest-neighbour (ANN) search possible via an HNSW index.

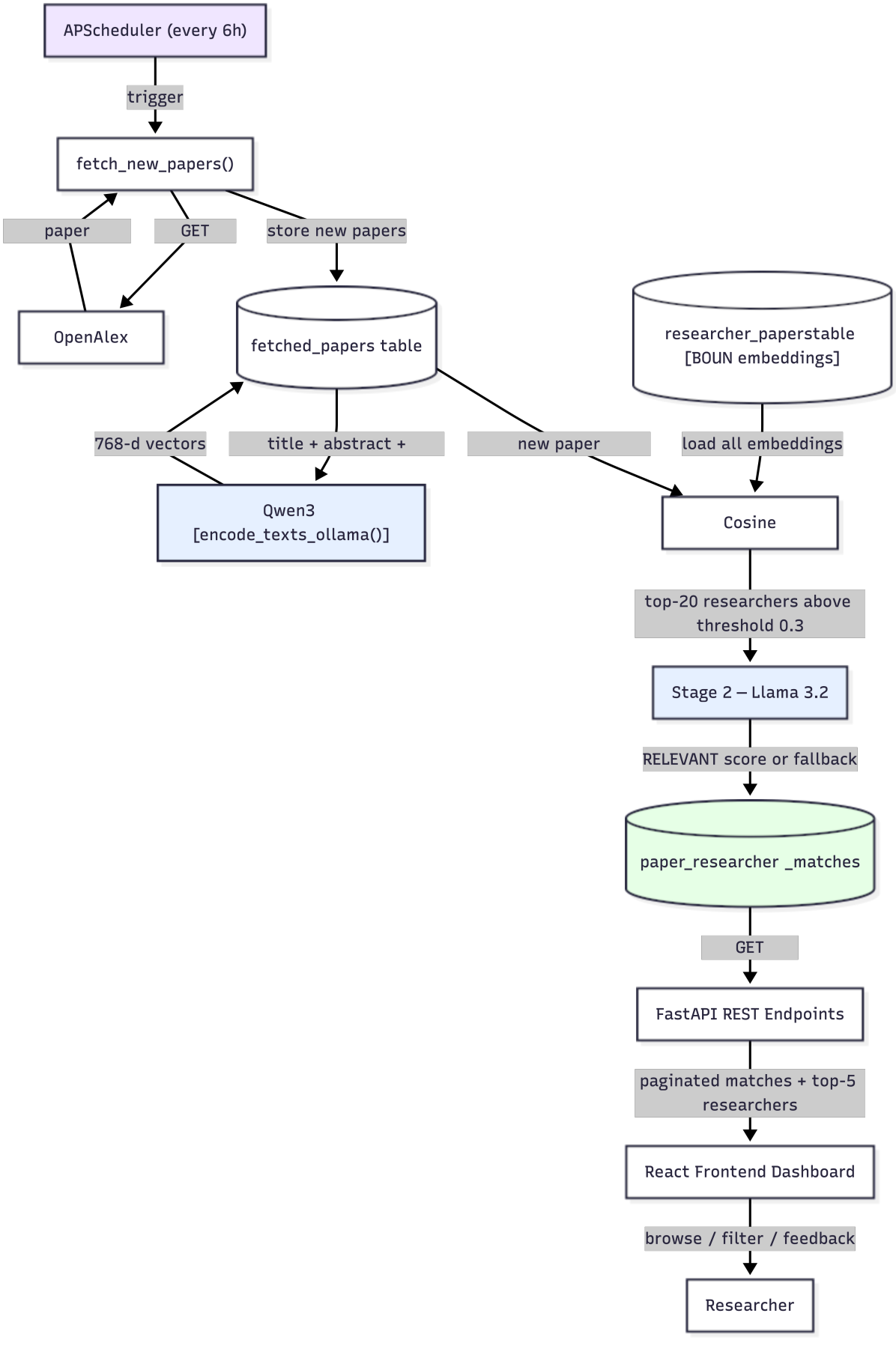


Figure 2.2. Data Flow Diagram — from OpenAlex ingestion through two-stage RAG matching to the recommendation feed.

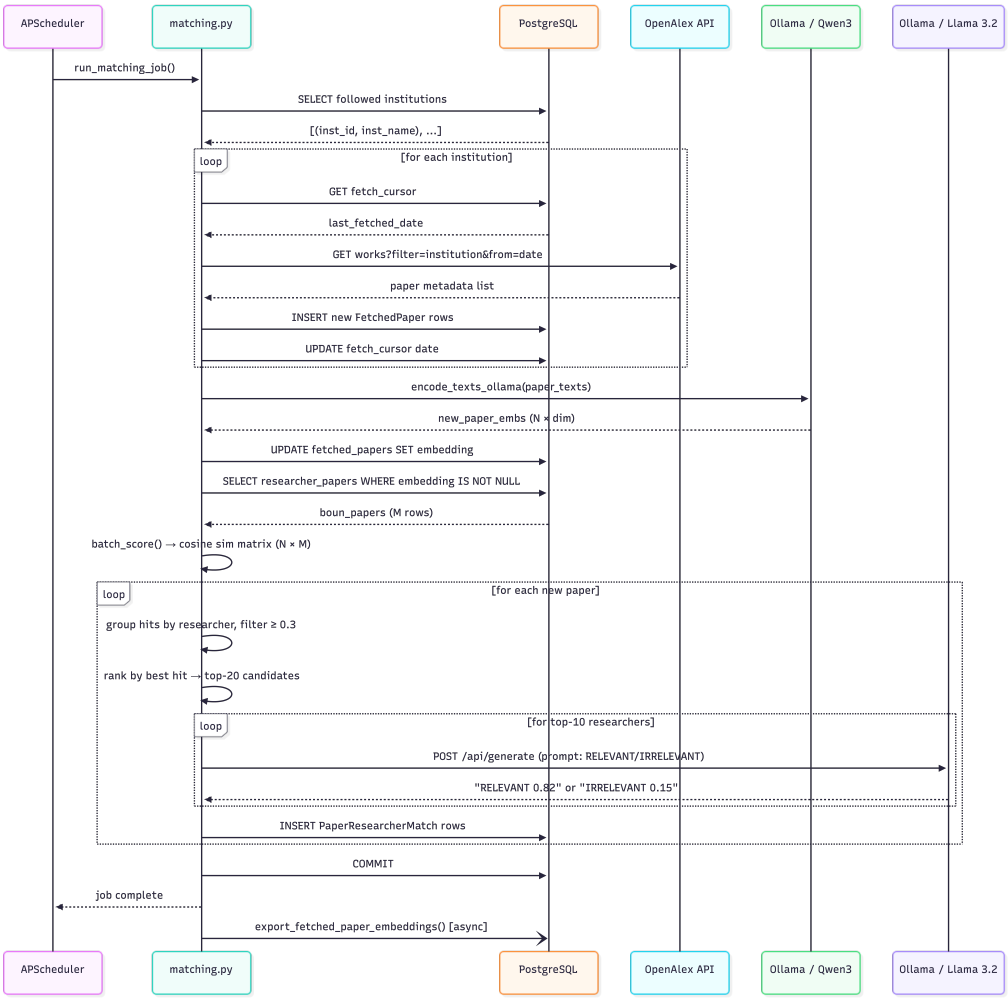


Figure 2.3. Sequence Diagram — Matching Job: Scheduler triggers paper fetch, Stage 1 embedding, Stage 2 LLM reranking, and match storage.

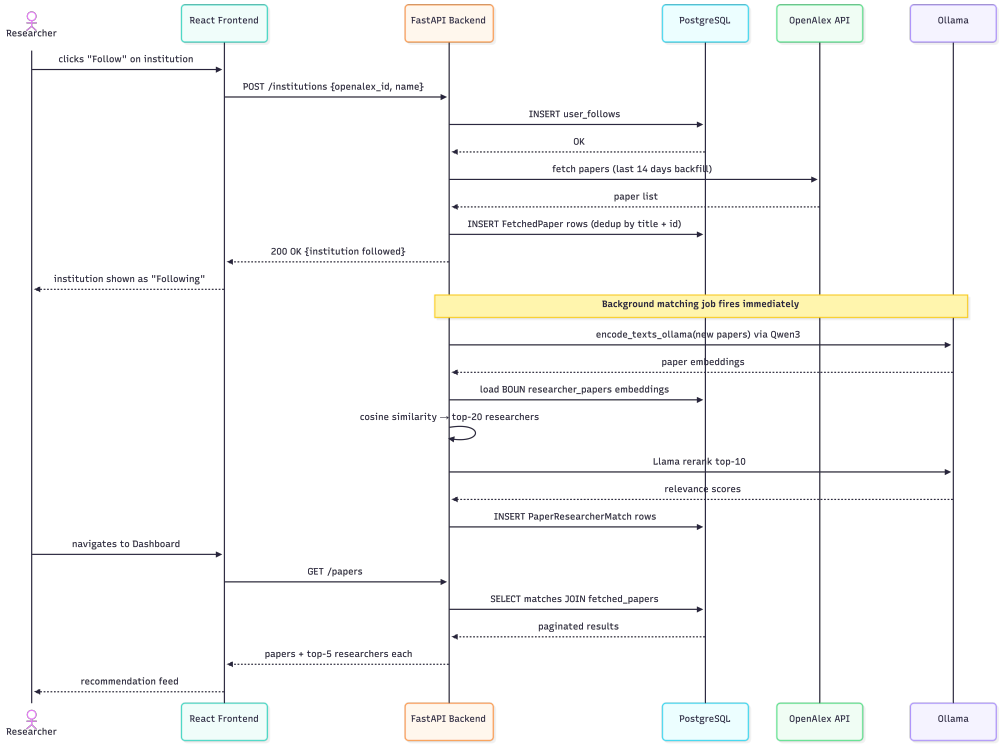


Figure 2.4. Sequence Diagram — Follow Institution: user follows an institution, triggering an immediate 14-day backfill and first-run matching.

2.3. Data Model

The key database entities are:

- `researchers` and `researcher_papers` — BOUN researcher profiles with per-paper embeddings stored as `vector(4096)` columns (pgvector).
- `fetched_papers` — Papers retrieved from followed institutions; embedding stored as `vector(4096)`.
- `paper_researcher_matches` — Cosine similarity scores linking fetched papers to BOUN researchers, including matched paper identifiers (JSON) and optional LLM rerank scores.
- `user_follows` and `fetch_cursors` — Per-user institution subscriptions and incremental fetch state.
- `paper_feedback` — User relevance feedback (thumbs up/down) on paper-researcher matches. Feedback is submitted via `POST /feedback`, persisted per user, and returned alongside each match in the paper feed so that the UI can reflect the user's prior judgement.

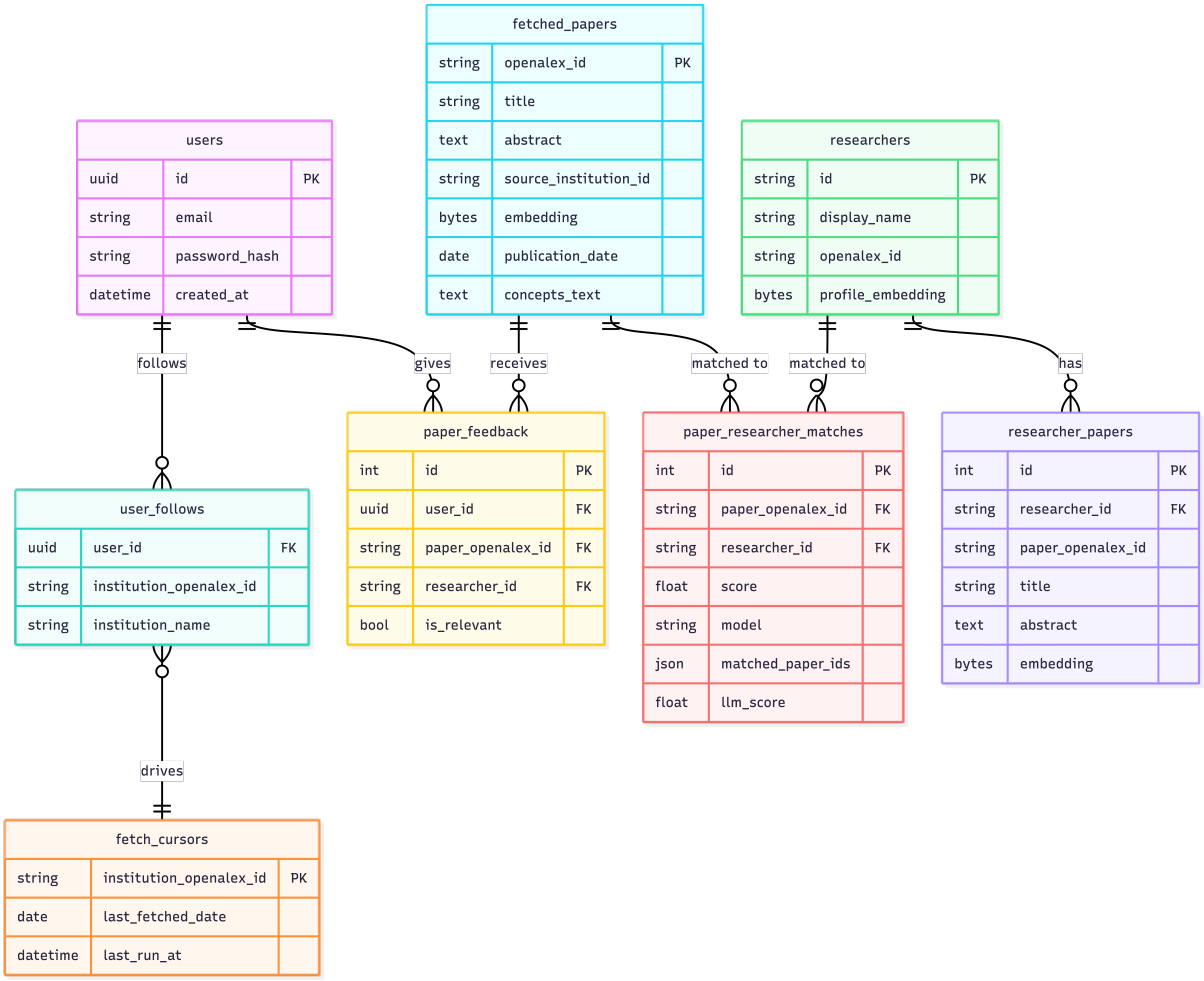


Figure 2.5. Entity–Relationship Diagram — core tables and their relationships.

2.4. Frontend Application

The frontend provides five protected views:

(i) **Dashboard** (`/`) — Paginated paper feed with top-5 matching BOUN researchers per paper, match scores, relevance feedback controls (thumbs up/down), and filter controls by institution, minimum match score, publication date range, full-text search over title and abstract (debounced, 400 ms), and concept/topic keyword filtering backed by OpenAlex concept tags (`concepts_text` ILIKE match).

(ii) **Institution Management** (`/institutions`) — Search the OpenAlex institution index, follow and unfollow institutions. Upon following, the backend immediately backfills the most recent papers.

(iii) **Researcher Directory** (`/researchers`) — Debounced full-text search over all BOUN researchers. Results are grouped by display name to merge duplicate OpenAlex identities into a single entry.

(iv) **Researcher Detail** (`/researchers/:id`) — A dedicated page for a single BOUN researcher showing all fetched papers that matched them, ordered by cosine similarity score, with pagination. Reachable from any researcher chip in the feed.

(v) **Admin Panel** (`/admin`) — Displays system statistics (researcher count, paper count, match count, number of unmatched papers, last job timestamp) and exposes two manual controls: trigger a full matching job and trigger a Stage-2 rematch for papers that were ingested without LLM reranking scores (e.g. when Ollama was unavailable during the original run).

Paper detail views are rendered as in-page modals rather than separate routes, reducing navigation overhead. Each modal synchronises its state with the URL via query parameters (`?paper=<id>` and `?researcher=<name>`), making individual papers and researcher profiles directly shareable by URL. A paper is marked as *seen* when its detail modal is opened; seen papers are visually dimmed in the feed so that users can distinguish reviewed from unreviewed recommendations.

Figures 2.6–2.7 show the main application screens.

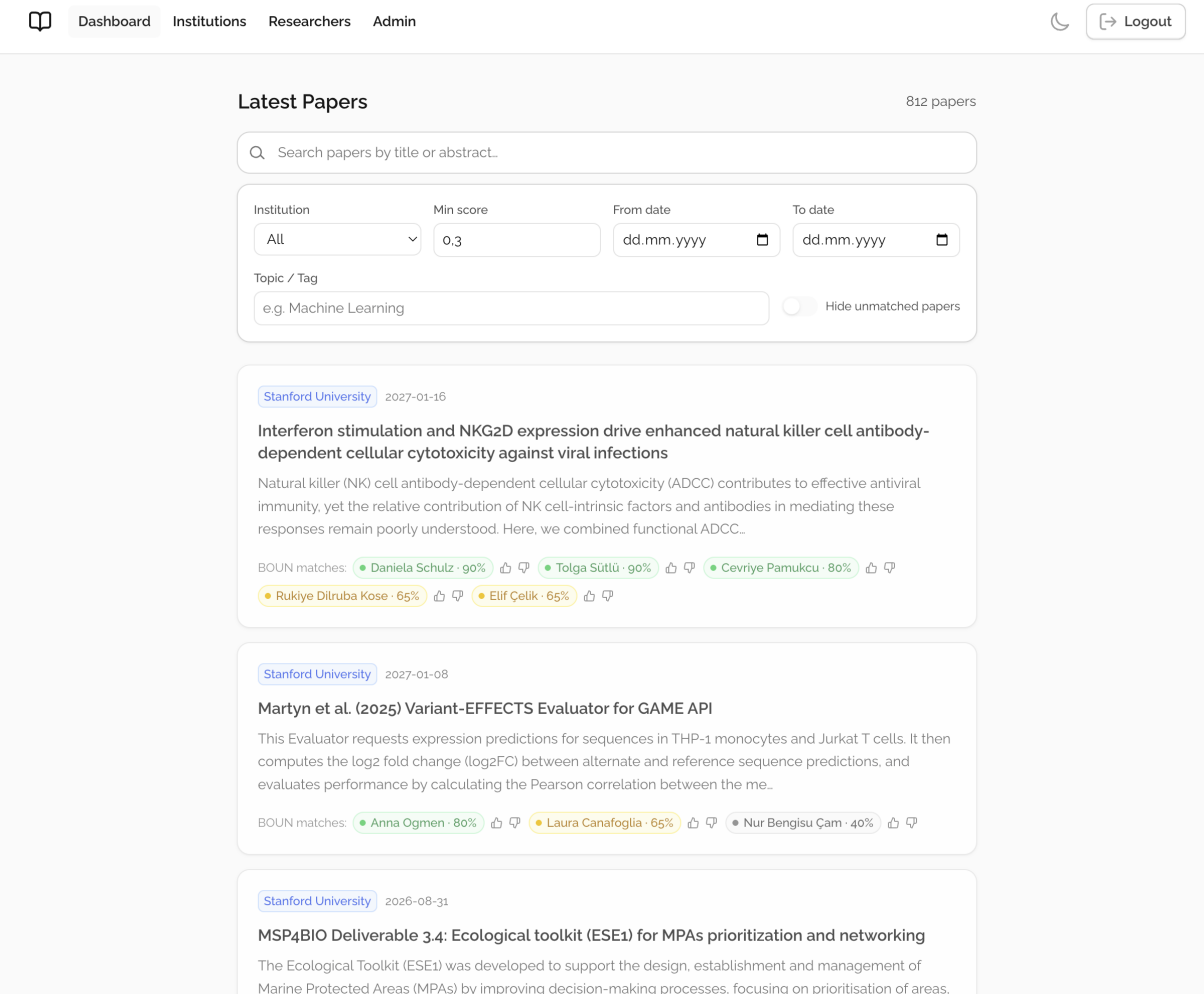


Figure 2.6. Dashboard — paginated paper feed with top-5 matching BOUN researchers, match scores, and seen/unseen state.

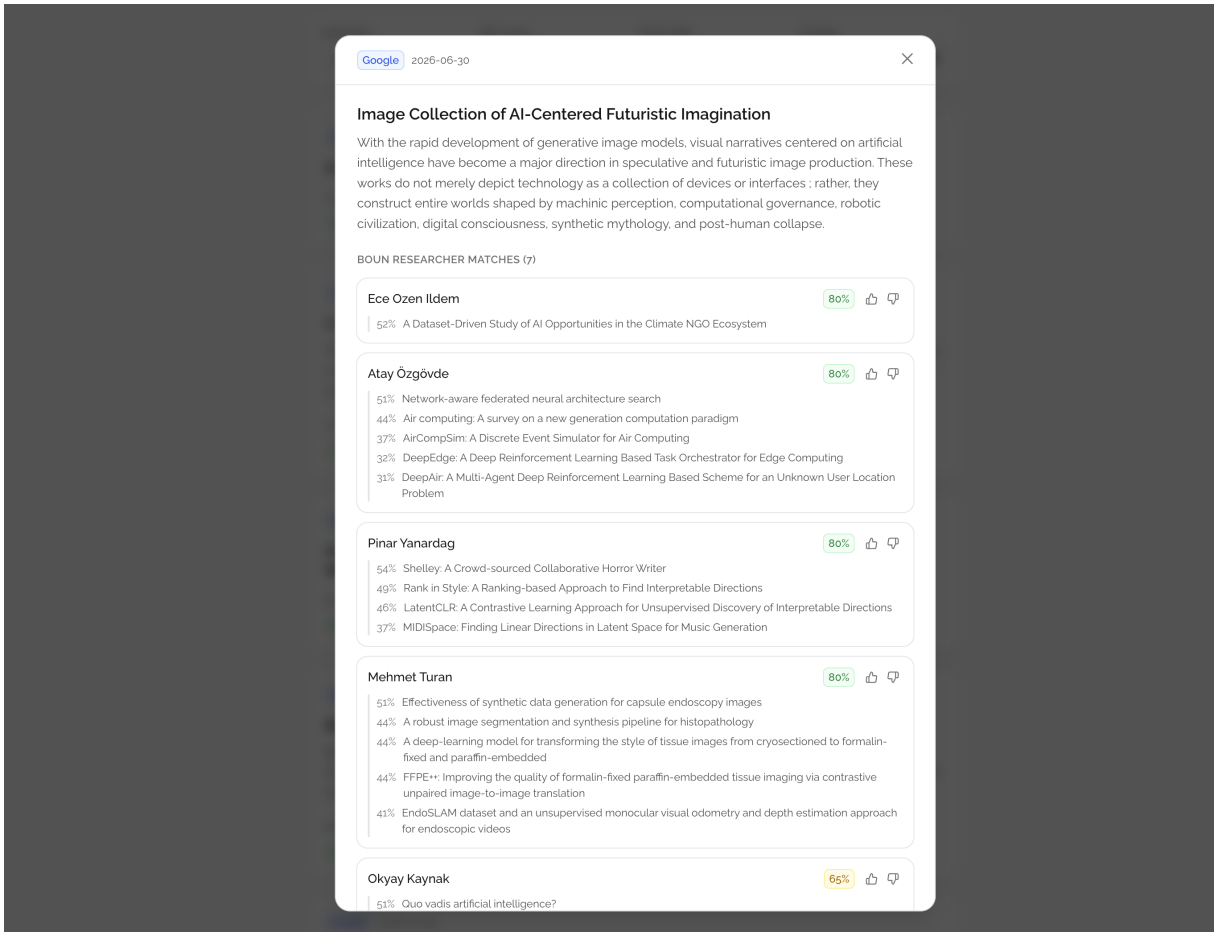


Figure 2.7. Paper Detail Modal — full abstract, concept tags, and matched researcher list with linked BOUN papers.

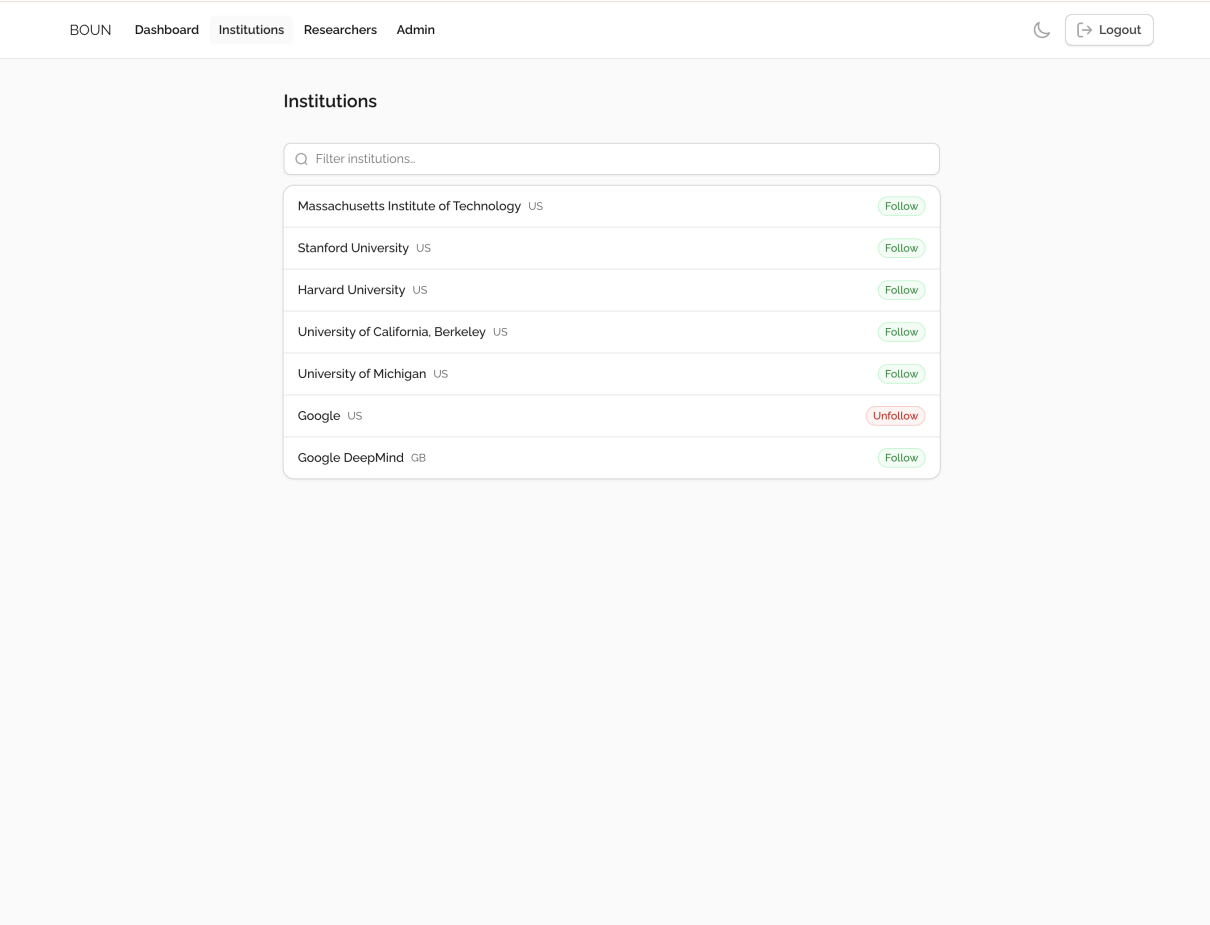


Figure 2.8. Institution Management — search, follow, and unfollow academic institutions.

3. System Requirements

3.1. Functional Requirements

(i) **Institution Following** — Authenticated users must be able to follow and unfollow academic institutions. Upon following, the system must backfill papers published within the past 14 days so that users receive an immediate set of recommendations.

(ii) **Automated Paper Ingestion** — The system must periodically fetch newly published papers from all followed institutions without manual intervention.

(iii) **Researcher-Paper Matching** — For each ingested paper, the system must compute similarity scores against all BOUN researcher profiles and store matches above a configurable threshold.

(iv) **Recommendation Display** — The user interface must display paginated recommendations showing the top-5 matching BOUN researchers per paper, along with the specific BOUN papers that contributed to the match.

(v) **Relevance Feedback** — Users must be able to mark a paper-researcher match as relevant or irrelevant; this feedback must be persisted per user.

(vi) **Filtering** — Users must be able to filter the recommendation feed by institution, publication date range, and minimum match score.

(vii) **Admin Controls** — Administrators must be able to view system statistics (including the count of papers without Stage-2 scores), manually trigger the full matching job, and selectively re-run Stage-2 LLM reranking for papers that were ingested without it.

(viii) **Seen Tracking** — The system must record which papers a user has viewed (via modal open) and reflect this in the feed so that previously reviewed papers are visually distinguished.

(ix) **Shareable Deep Links** — Paper and researcher detail views must be accessible via stable URLs (`?paper=<id>`, `?researcher=<name>`) so that links can be shared directly.

3.2. Non-Functional Requirements

(i) **Graceful Degradation** — If the local Ollama instance is unavailable, the system must fall back to Stage 1 embedding scores rather than failing the job.

(ii) **Incremental Fetching** — The system must track the last fetch date per institution using a cursor mechanism to avoid re-fetching previously processed papers. On first follow, the cursor is initialised to 90 days prior to enable immediate backfill.

(iii) **Embedding Persistence** — All computed embeddings must be stored as serialized binary arrays in the database to avoid redundant GPU computation across restarts.

(iv) **Asynchronous Processing** — All database operations must be non-blocking; long-running embedding computations must be offloaded to thread pools.

(v) **Configurability** — Match threshold and top-K parameters must be configurable via environment variables without code changes.

(vi) **Cloud Deployability** — The system must be able to operate without a local GPU or Ollama instance; external embedding API providers must be tried automatically via a fallback chain so that the matching job succeeds in cloud environments.

4. Research Methodology and Evaluation

This chapter describes the offline research component, which drives the model selection decisions made in the production system. The research was conducted across three weekly phases.

4.1. Dataset Construction

4.1.1. Data Collection

All paper metadata was retrieved from the OpenAlex API. Two corpora were constructed:

- **BOUN Corpus**: 5,038 papers fetched for BOUN-affiliated authors; 4,608 retained after filtering for English-language papers with non-empty abstracts.
- **Followed Institutions Corpus**: 10,000 papers fetched from the seven target institutions (post-2020); 9,946 retained after the same filter.

4.1.2. Preprocessing

OpenAlex returns paper abstracts in an *inverted index* format (a mapping from tokens to their character positions). A reconstruction step converts this back to readable prose. Additional preprocessing includes HTML and URL removal, stopword filtering, and extraction of OpenAlex concept tags, which are concatenated to the title and abstract to form the final query text. The preprocessing pipeline is implemented in `research/src/preprocessing.py`.

4.1.3. Benchmark Dataset for Cited Paper Ranking

A benchmark dataset was constructed to evaluate how well each method ranks truly relevant papers above random negatives. The construction procedure (implemented in `research/cited_paper_ranking.ipynb`) is:

(i) Select papers from the followed-institution corpus that cite between 6 and 14 references also present in the corpus. This yields 174 *query papers* (P_i).
(ii) For each P_i with n cited references: collect the n references as *positives*, and sample $2n$ random papers from the corpus (excluding references and same-author works) as *negatives*, giving $3n$ total candidates.
(iii) Cache evaluation sets to `data/results_cache/week3/eval_sets.pkl`.

Query text statistics across the 174 papers: mean length 1,434 characters, median 838 characters.

For quantitative evaluation, 100 queries were randomly sampled from the 174 using a fixed random seed (`RANDOM_SEED = 42`) to ensure reproducibility. The 100-query subset was chosen to balance evaluation cost — the Llama reranker processes each candidate sequentially via a generative LLM call, making full evaluation over all 174 queries pro-

hibitively slow — while maintaining sufficient statistical coverage across the distribution of reference counts ($n = 6$–14).

4.1.4. Pipeline Validation

Before the main benchmark evaluation, two sanity checks were conducted on the BOUN corpus to confirm that the similarity pipeline correctly ranks known-relevant papers above random distractors. Both tasks were implemented in `research/similarity_evaluation.ip` using three methods: TF-IDF, BM25, and MiniLM.

Task 3a — Same-Author Paper Ranking.. For 20 randomly sampled prolific BOUN authors (each with $\geq$ 10 papers in the corpus), one paper per author was selected as the query. The remaining papers by the same author were treated as positives, and 50 randomly drawn papers were added as negatives to form the retrieval corpus. The metric is the average rank position of same-author papers and the count of same-author papers appearing in the top-5 retrieved results.

Table 4.1. Same-author paper ranking: average rank and top-5 hit count (out of 5 same-author papers) across 20 BOUN authors.

Method	**Avg Same-Author Rank**	**Same-Author in Top-5**
TF-IDF	13.9	4.70
BM25	15.5	4.80
MiniLM	13.1	4.90

All three methods place on average ≈ 4.8 of the 5 same-author papers in the top-5 retrieved results, confirming that the pipeline reliably clusters a researcher's output together. MiniLM achieves the best average rank (13.1) despite the relatively large retrieval corpus.

Task 3b — Cited Paper Ranking (BOUN intra-corpus).. For 85 BOUN papers that cite at least 3 other BOUN papers, each paper was used as a query and its in-corpus references were treated as positives. Random non-cited BOUN papers were added as negatives. The task measures how highly each method ranks the true references.

Table 4.2. Cited-paper retrieval on the BOUN intra-corpus (85 queries, ≥ 3 in-corpus references each).

Method	Avg Cited Rank	Cited in Top-5
TF-IDF	4.0	3.65
BM25	6.7	3.45
MiniLM	3.8	3.55

TF-IDF and MiniLM both place cited papers at an average rank of ≈ 4, recovering 3.5–3.7 out of the available references in the top-5. BM25 performs somewhat worse (average rank 6.7), suggesting that its term-frequency saturation is less suited to the short, abstract-heavy text representations used here. These results validate the pipeline design and motivated the more rigorous cross-institutional benchmark in Section 4.1.3.

4.2. Text Representation and Similarity Methods

Nine distinct methods were implemented and evaluated, covering sparse retrieval baselines, dense neural embeddings, and LLM-based reranking. All implementations are in `research/src/similarity.py`[1] .

- **BM25** [7] — Probabilistic term-frequency ranking using the Okapi BM25 formula with saturation normalization. No GPU required.
- **TF-IDF** — Scikit-learn `TfidfVectorizer` (50,000 features) with cosine similarity. No GPU required.
- **MiniLM** [6] — `all-MiniLM-L6-v2` sentence transformer; 384-dimensional embeddings, batch size 256.
- **SPECTER2** [1] — `allenai/specter2_base`; a transformer pretrained on citation graphs for scientific paper representation; 768-dimensional embeddings, batch size 128.
- **embeddinggemma** — Google Gemma embedding model served via Ollama.
- **Qwen** — `qwen3-embedding:latest` via Ollama; high-dimensional dense embeddings from the Qwen3 model family [8].
- **Llama reranker** — Llama 3.2 3B [2] via Ollama used as a *reranker*: given a query paper and all candidates, the model is prompted to score each candidate's relevance and return a ranked list.
- **llama+minilm** — Late fusion: min-max normalized average of Llama reranker scores and MiniLM cosine similarities.
- **qwen+minilm** — Late fusion: L2-normalized concatenation of Qwen and MiniLM embeddings, followed by cosine similarity.

[1]Available at `https://github.com/senaoz/SWE-599`/blob/main/research/src/similarity.py

4.2.1. Field Ablation

To determine the optimal text representation for Qwen3 retrieval, a systematic ablation was conducted over four input field combinations on the cited paper ranking task (100 queries). The four combinations are: *Title only*, *Abstract only*, *Title + Abstract*, and *Title + Abstract + Concepts*, where "Concepts" refers to the OpenAlex concept tags appended as a comma-separated list. All configurations use identical Qwen3 embeddings; only the text input changes.

Table 4.3 reports MRR, hit rate at $n\%$, and nDCG@n, averaged over the 100-query evaluation set.

Table 4.3. Field ablation with Qwen3 embeddings: MRR, HR@$n\%$, and nDCG@n across four text field combinations (100 queries, cited paper ranking task).

Fields	**MRR**	**HR@$n\%$**	**nDCG@n**
Title only	0.493	0.339	0.324
Abstract only	**0.684**	**0.460**	**0.474**
Title + Abstract	0.613	0.437	0.434
Title + Abstract + Concepts	0.485	0.339	0.324

Three findings emerge from this ablation:

- **The abstract dominates.** Switching from title-only to abstract-only raises MRR from 0.493 to 0.684 (+39%) and HR from 0.339 to 0.460 (+36%). The abstract encodes the full methodological and topical content of a paper; titles provide only a compressed label. For a high-dimensional dense encoder like Qwen3, the richer semantic content of the abstract is far more discriminative.
- **Adding the title to the abstract marginally reduces performance.** Title + Abstract (MRR 0.613) underperforms Abstract only (MRR 0.684, −10%). Titles introduce short, high-frequency phrase fragments that can dilute the abstract's signal in the embedding space rather than adding independent information.
- **Concept tags hurt Qwen3 performance.** Adding OpenAlex concept tags reduces MRR from 0.613 back to 0.485 — nearly as low as title-only. Unlike sparse models such as TF-IDF (where concept tags improve term overlap), Qwen3 already captures domain semantics internally through its dense representations. Appending a flat list of concept labels shifts the embedding toward a topic-classification signal and away from the fine-grained retrieval signal encoded in the abstract, degrading ranking quality.

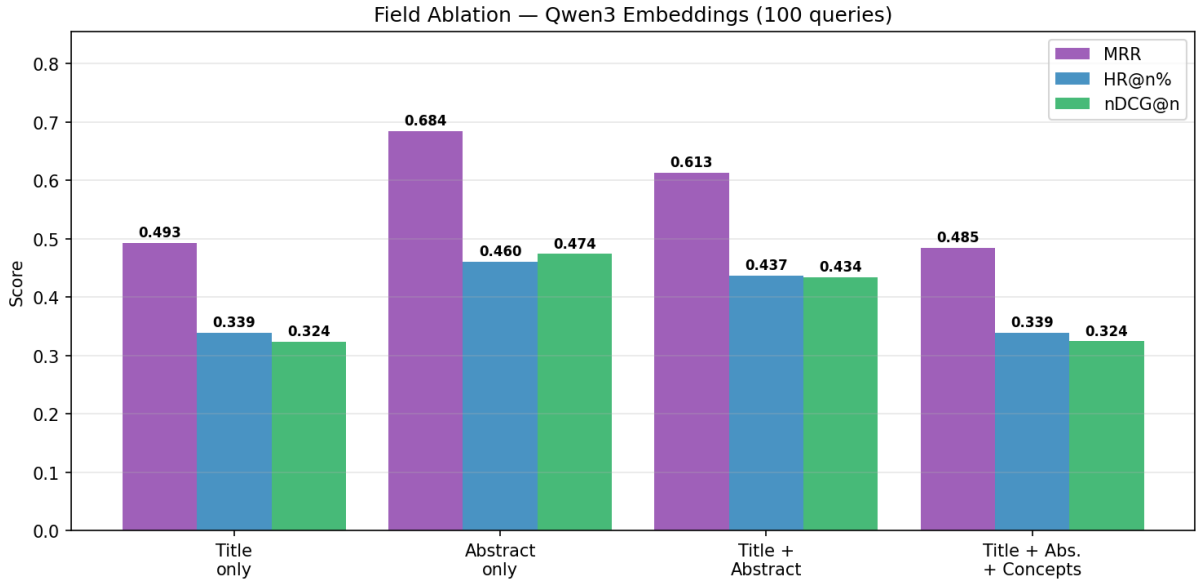


Figure 4.1. Field ablation with Qwen3 embeddings: MRR, HR@n%, and nDCG@n across four text field combinations (100 queries, cited paper ranking task).

These results show that abstract-only is the strongest text representation for Qwen3. The production system currently uses Title + Abstract + Concepts; switching to abstract-only embeddings is identified as a concrete future improvement (see Section 5).

4.2.2. Pipeline Component Ablation

To quantify the contribution of each stage in the two-stage RAG pipeline, a component ablation was conducted comparing three retrieval configurations on the cited paper ranking task (100 queries, fixed candidate pool of $3n$ candidates per query):

(i) **Stage 1 only (Qwen):** Dense embedding retrieval with Qwen3, ranking all $3n$ candidates by cosine similarity.

(ii) **Stage 2 only (Llama reranker):** LLM reranking of the full $3n$ candidate set without any prior embedding filter.

(iii) **Two-stage (Qwen → Llama):** Qwen embedding retrieval selects the top candidates; Llama reranks only that shortlist.

Table 4.4. Pipeline component ablation: hit rate, MRR, nDCG@10, and Top-1 accuracy for three pipeline configurations (100 queries).

Configuration	**Hit Rate%**	**MRR**	**nDCG@10**	**Top-1%**
Stage 1 only (Qwen)	37.42	0.513	0.642	25
Two-stage (Qwen → Llama)	36.98	0.589	0.681	38
Stage 2 only (Llama)	**44.99**	**0.841**	**0.838**	**75**

The results reveal a recall-precision tradeoff inherent in cascaded retrieval:

- **Stage 2 alone is the strongest retriever on a fixed candidate pool.** When Llama is given all $3n$ candidates, it achieves MRR 0.841 and Top-1 accuracy of 75%, substantially outperforming Qwen-only (MRR 0.513, Top-1 25%) and the two-stage pipeline.
- **Cascading improves ranking precision but introduces recall loss.** The Qwen → Llama pipeline improves MRR over Stage 1 alone by +15% (0.513 → 0.589) and Top-1 accuracy by +52% (25% → 38%), because Llama re-orders Qwen's top candidates more accurately. However, its overall hit rate (36.98%) falls slightly below Qwen-only (37.42%) — a direct consequence of recall loss: true positives excluded by the Qwen filter in Stage 1 cannot be recovered in Stage 2.
- **Production constraints justify the cascade.** The research benchmark evaluates retrieval over a fixed pool of 18–42 candidates per query. In production, Stage 1 must score each new paper against thousands of BOUN researcher papers; applying the Llama reranker to all of them per incoming paper would be computationally prohibitive. The two-stage architecture therefore accepts an approximately 8 percentage point hit-rate cost relative to the oracle setting in exchange for practical throughput — a tradeoff that native `pgvector` cosine search makes efficient at the database level.

4.3. Evaluation Metrics

Three metrics are computed for each query, implemented in `research/src/metrics.py`:

- **Hit Rate @ n%** — The fraction of the top-n ranked candidates that are true references, where n equals the number of positives for that query.
- **Mean Reciprocal Rank (MRR)** — The reciprocal of the rank position of the first true positive, averaged across queries.
- **nDCG @ n** — Normalized Discounted Cumulative Gain with binary relevance, evaluated at position n.

4.4. Results

Table 4.5 reports mean and standard deviation of all three metrics across 100 sampled queries, ranked by mean hit rate. Top-1 accuracy (the fraction of queries where the first-ranked candidate is a true reference) is also reported.

Table 4.5. Cited paper ranking results across 9 methods (100 queries, $n = 6$–14 references per query).

Method	**Hit Rate %**	**Std**	**MRR**	**Std**	**nDCG@n**	**Top-1**
Llama reranker	**44.99**	18.87	**0.841**	0.285	**0.838**	**75%**
Qwen	37.42	14.39	0.513	0.303	0.642	25%
Qwen+MiniLM	37.22	15.99	0.526	0.331	0.640	30%
MiniLM	35.90	16.70	0.536	0.345	0.632	33%
Llama+MiniLM	34.50	15.20	0.522	0.333	0.633	30%
embeddinggemma	33.95	15.95	0.471	0.328	0.603	25%
TF-IDF	33.41	14.62	0.536	0.336	0.629	32%
SPECTER2	30.21	15.83	0.465	0.322	0.582	24%
BM25	23.09	11.70	0.410	0.304	0.528	18%

4.5. Discussion

The most striking finding is that the Llama 3.2 3B reranker substantially outperforms all dense embedding methods. With a mean hit rate of 44.99% and a top-1 accuracy of 75%, it outperforms the second-best method (Qwen embeddings) by over 7 percentage points in hit rate and 50 percentage points in top-1 accuracy. The MRR of 0.841 indicates that, on average, the first true reference appears within the top two ranked candidates.

This result is notable because Llama 3.2 3B is a relatively small model (3 billion parameters). Its advantage over dense embeddings likely stems from its ability to reason about semantic relevance across all candidates in a single generation pass, rather than independently encoding each text into a fixed-dimensional vector.

The pipeline validation results (Section 4.1.4) confirm that even the weakest method (BM25) reliably clusters same-author papers, so the performance gap between methods on the harder cross-institutional cited-paper task reflects genuine differences in semantic understanding rather than implementation artefacts.

A second surprising finding is that SPECTER2 — a model specifically pretrained

on the citation graph of scientific papers — underperforms the general-purpose MiniLM model and even TF-IDF on this task. We hypothesize that the cited paper ranking task, which requires fine-grained lexical and conceptual matching within a narrow candidate set, does not benefit from SPECTER2's global citation-structure pretraining objective.

Combining embeddings (Qwen+MiniLM, Llama+MiniLM) provides marginal gains over MiniLM alone but does not close the gap with the Llama reranker.

These results informed the production system's design: Stage 1 uses Qwen dense embeddings for efficient candidate retrieval from the full BOUN researcher corpus, and Stage 2 uses Llama 3.2 3B to rerank the top-10 candidate researchers via natural language reasoning. The deployed system validates this two-stage design in practice, with Stage 1 narrowing the candidate set efficiently using the native `pgvector` cosine operator and Stage 2 applying LLM reasoning only to the shortlisted candidates.

5. Conclusion and Future Work

This report presents the complete design and implementation of an NLP-based academic paper recommendation system for BOUN researchers.[2] The system is fully operational end-to-end: users can follow institutions, receive paginated paper recommendations matched to BOUN researcher profiles, explore researcher detail pages, and provide relevance feedback. The backend runs a two-stage RAG pipeline every six hours, combining Qwen embedding retrieval with Llama 3.2 3B reranking.

The offline research evaluation confirms that a local LLM reranker significantly outperforms dense and sparse embedding baselines on the cited paper ranking task, motivating the two-stage production architecture. Nine methods were compared across 100 evaluation queries using hit rate, MRR, and nDCG. The field ablation further reveals that abstract-only text is the strongest input representation for Qwen3 embeddings — contrary to the intuition that more fields are always better — because concept tags shift the embedding signal away from the fine-grained semantic content of the abstract.

Beyond the initial implementation, two significant engineering improvements were completed after the progress report milestone:

- **pgvector migration** — Embedding storage was migrated from `BYTEA` binary blobs to native `vector(4096)` PostgreSQL columns via the `pgvector` extension. This

[2]Source code and deployment instructions: `https://github.com/senaoz/SWE-599`

eliminates full-table Python-side matrix multiplication for similarity search and makes SQL-level ANN queries feasible.

- **Embedding provider fallback chain** — The system was extended with a two-tier fallback (local Ollama → Ollama Cloud), making the matching job functional in environments where a local Ollama instance is unreachable.

Remaining future work includes:

- **Switch Stage 1 to abstract-only embeddings.** The field ablation (Section 4.2.1) shows that abstract-only text achieves MRR 0.684 with Qwen3, compared to MRR 0.485 for the current Title + Abstract + Concepts configuration — a 41% improvement. Updating the embedding input in both the seeder and the matching job would directly improve retrieval quality with no architectural changes.
- Create an HNSW index on the `researcher_papers.embedding` column to accelerate Stage 1 retrieval at scale (`pgvector` infrastructure is already in place).
- Add DashScope Qwen3-Embedding support once the provider exposes a model with 4,096-dimensional output compatible with the current schema.
- Surface user relevance feedback (already persisted in `paper_feedback`) to personalize match thresholds per researcher.
- Integrate email notifications to alert researchers when high-scoring matches are found.
- Add a test suite (unit and integration) and a CI/CD pipeline.
- Add monitoring and structured logging for production observability.

Bibliography

[1] Arman Cohan et al. "SPECTER: Document-level Representation Learning using Citation-informed Transformers". In: *CoRR* abs/2004.07180 (2020). arXiv: 2004.07180. URL: https://arxiv.org/abs/2004.07180.

[2] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs.AI]. URL: https://arxiv.org/abs/2407.21783.

[3] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *CoRR* abs/2005.11401 (2020). arXiv: 2005.11401. URL: https://arxiv.org/abs/2005.11401.

[4] Jason Priem, Heather Piwowar, and Richard Orr. *OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts*. 2022. arXiv: 2205.01833 [cs.DL]. URL: https://arxiv.org/abs/2205.01833.

[5] Sebastián Ramírez. *FastAPI*. 2018. URL: https://fastapi.tiangolo.com.

[6] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019, pp. 3982–3992.

[7] Stephen Robertson and Hugo Zaragoza. "The probabilistic relevance framework: BM25 and beyond". In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.

[8] An Yang et al. *Qwen3 Technical Report*. 2025. arXiv: 2505.09388 [cs.CL]. URL: https://arxiv.org/abs/2505.09388.